Clase 0: Programación, Lenguajes y Python

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Contents

1. Introducción a la programación
2. Interacción con el PC
3. La consola
4. Compilación de código
5. Python

Introducción a la

Qué es programar

Hacer que el computador haga algo a partir de instrucciones

- Instrucciones: *lenguaje de programación*
- Ejs:
  - Web (HTML,CSS,PHP,JS)
  - Apps (Java)
  - Ciencia (Python,R,Julia,MATLAB)
  - Esta presentación (LaTeX)

El foco del curso estará en programación científica

Qué es programar

Hacer que el computador haga algo a partir de instrucciones

- Instrucciones: *lenguaje de programación*
- Ejs:
  - Web (HTML,CSS,PHP,JS)
  - Apps (Java)
  - **Ciencia (Python,R,Julia,MATLAB)**
  - Esta presentación (LaTeX)

El foco del curso estará en programación científica

Lecturas muy recomendadas

https://swcarpentry.github.io/python-novice-inflammation/
https://rosalind.info
+ internet y LLMs

Ejemplo: Artritis

Imaginen que tenemos una cura milagrosa para la artritis que actúa en 3 semanas.

- Seguimiento pacientes

(Dosis después de primer síntoma)

- Datos: Nro de inflamaciones por día
- Visualización
- Estadística $\rightarrow$ Concluir!

Ejemplo: Artritis

Imaginen que tenemos una cura milagrosa para la artritis que actúa en 3 semanas.

- Seguimiento pacientes
- Datos: Nro de inflamaciones por día

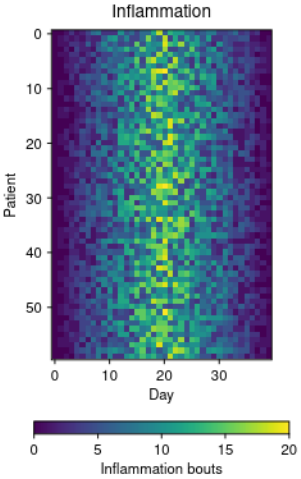
```
0,0,1,3,1,2,4,7,8,3,3,3,10,5,7,4,7,7,12,
0,1,2,1,2,1,3,2,2,6,10,11,5,9,4,4,7,16,8
0,1,1,3,3,2,6,2,5,9,5,7,4,5,4,15,5,11,9,
```

- Visualización
- Estadística $\rightarrow$ Concluir!

Ejemplo: Artritis

Imaginen que tenemos una cura milagrosa para en 3 semanas.

- Seguimiento pacientes
- Datos: Nro de inflamaciones por día
- Visualización



Ejemplo: Artritis

Imaginen que tenemos una cura milagrosa para la artritis que actúa en 3 semanas.

- Seguimiento pacientes
- Datos: Nro de inflamaciones por día
- Visualización
- Estadística $\rightarrow$ Concluir!

Ej II: Pensamiento algorítmico

Cómo salgo de esta sala? (avanzar un paso, girar)

Y si solo sé girar a la derecha?

Ej II: Pensamiento algorítmico

Cómo salgo de esta sala? (avanzar un paso, girar)

Y si solo sé girar a la derecha?

Ej II: Lo que hicimos

- Considerar un pequeño conjunto de instrucciones
- Generar nuevas instrucciones más complejas (como la célula!)
- Combinar instrucciones para un objetivo

Contents

1. Introducción a la programación
2. Interacción con el PC
3. La consola
4. Compilación de código
5. Python

Interacción con el PC

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Unix)
- iOS (FreeBSD, Darwin)
- Linux (Debian, Ubuntu, Fedora, ...)
- Android
- z/OS
- Linux zOS
- RTOS, Nuttx

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

El OS coordina funcionamiento del computador.

- Windows (MS-DOS)
- Mac OS (BSD Darwin)
- BSD (FreeBSD, OpenBSD)
- GNU/Linux (Debian, Ubuntu, Fedora, ...)
- Android
- iOS
- Symbian
- BeOS, Haiku

Sistemas operativos

Data span:
Trending past 12 months Go

Rank	Distribution	HPD*
1	MX Linux	2475▼
2	Mint	2208▲
3	EndeavourOS	1780▼
4	Debian	1294▼
5	Ubuntu	1185▼
6	Manjaro	1173▲
7	Fedora	1027▬
8	Pop!_OS	839▲
9	openSUSE	810▼
10	Zorin	809▬

Contents

1. Introducción a la programación
2. Interacción con el PC
3. La consola
4. Compilación de código
5. Python

La consola

Principio

- La consola (Unix) es una forma de interacción con el PC
- Funciona a través de programas
- Las interfaces gráficas usan consola!

La consola: Inicio

La consola Bash en Linux permite interactuar con el PC[1]

```
nico@nico-mini:~/Downloads$
```

- `nico`: Usuario
- `nico-mini`: hostname, nombre de red del PC
- `~/Downloads`: Carpeta actual
- `~`: Carpeta 'home'

[1] Se puede instalar en Windows con WSL, queda para ayudantía

Sistema de archivos

En Linux, los archivos funcionan como uno espera:

- Carpeta 1
  - Archivo 1
  - Archivo 2
- Carpeta 2
  - Carpeta 3
  - Archivo 3
- ...

Navegación básica en Bash

- `ls`: Mostrar contenido de carpeta
- `cd`: Cambiar de directorio
- `mkdir`: Crear carpeta
- `rm`: Eliminar archivo o carpeta
- `cp`: Copiar archivo o carpeta
- `nano`: Editor de texto
- `cat`: Mostrar archivo

Pro-tip: Autocompletar con `tab`

Concepto de programa

Ver contenidos de 'Documents':

$$\$ \quad \underbrace{\texttt{ls}}_{\text{Comando}} \quad \underbrace{\textit{Documents}}_{\text{Argumento}}$$

Argumentos y opciones

$$\$ \quad \underbrace{\texttt{ls}}_{\text{Comando}} \quad \underbrace{\texttt{-l -a --color}}_{\text{Opciones}} \quad \underbrace{\textit{Documents}}_{\text{Argumento}}$$

La más útil: `--help/-h`.

Veamos...

Contents

1. Introducción a la programación
2. Interacción con el PC
3. La consola
4. Compilación de código
5. Python

Compilación de código

La compilación

Consiste en traducir nuestro lenguaje al del PC

- Lo que nosotros entendemos: Lenguaje
- Lo que entiende el computador: 1000101010101101010110101101010110110101011101010000

La compilación

Consiste en traducir nuestro lenguaje al del PC

- Lo que nosotros entendemos: Lenguaje
- Lo que entiende el computador: 1000101010101101010110101101010110110101011101010000

Ejemplo de compilación

Para humanos:

```c
#include <stdio.h>
   int main() {
   printf("Hello, World!\n");
   return 0;
            }
```

Para PC:

```
    .file    "hello.c"
    .text
    .section .rodata
.LC0:
    .string  "Hello, World!"
    .text
    .globl   main
    .type    main, @function
main:
```

El trabajo de compilación

1. Procesar el archivo con código
2. Generar símbolos de programas
3. Juntar símbolos para generar programa

Pros	Contras
Eficiente	Inflexible

Ejemplos de programas compilados: `ls`, `cd`, ...

El trabajo de compilación

1. Procesar el archivo con código
2. Generar símbolos de programas
3. Juntar símbolos para generar programa

Pros	Contras
Eficiente	Inflexible

Ejemplos de programas compilados: `ls`, `cd`, ...

El trabajo de compilación

1. Procesar el archivo con código
2. Generar símbolos de programas
3. Juntar símbolos para generar programa

Pros	Contras
Eficiente	Inflexible

Ejemplos de programas compilados: `ls`, `cd`, ...

Alternativa

Lenguajes compilados son eficientes pero...

- Más complejos
- Requieren recompilar constantemente

Alternativa: **Lenguajes interpretados**

- Muchos subprogramas pre-compilados
- No se compila, se ejecuta secuencialmente (una línea a la vez)
- Permiten una consola interactiva

Alternativa

Lenguajes compilados son eficientes pero...

- Más complejos
- Requieren recompilar constantemente

Alternativa: **Lenguajes interpretados**

- Muchos subprogramas pre-compilados
- No se compila, se ejecuta secuencialmente (una línea a la vez)
- Permiten una consola interactiva

Alternativa

Lenguajes compilados son eficientes pero...

- Más complejos
- Requieren recompilar constantemente

Alternativa: **Lenguajes interpretados**

- Muchos subprogramas pre-compilados
- No se compila, se ejecuta secuencialmente (una línea a la vez)
- Permiten una consola interactiva

Algunos ejemplos

Compilados

- C/C++/C#
- Java
- Rust
- Julia

Interpretados

- Python
- R
- MATLAB

Ranking de lenguajes (TIOBE Ranking 2026)

1. Python (21.81%)
2. C (11.05%)
3. C++ (8.55 %)
4. Java (8.12 %)
5. C# (6.83 %)
6. Javascript (2.92 %)
7. Visual Basic (2.85 %)
8. R (2.19%)
9. SQL (1.93%)
10. Delphi / Object Pascal (1.88%)

Contents

1. Introducción a la programación
2. Interacción con el PC
3. La consola
4. Compilación de código
5. Python

Python

Datos sobre Python

- Lenguaje interpretado, orientado a objetos
- Creado en 1991 por Guido van Rossum (en navidad...)
- Basado en C
- Librerías de IA: PyTorch, Keras, TensorFlow
- MUCHAS librerías científicas
- Ejecuta extensión `'.py'`

[...] as a hobby project to keep himself busy during the Christmas holidays of 1989.

Cómo usar Python

1. Como programa

```
nico@nico-pc:~/Codes$ python test.py
```

2. Como consola interactiva
3. **Jupyter/Collab/IDE (ayudantía)**

Veamos la consola

Cómo usar Python

1. Como programa
2. Como consola interactiva

```
nico@nico-pc:~/Codes$ python
Python 3.10.12 (main, Mar 22 2024, 16:50:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 
```

3. Jupyter/Collab/IDE (ayudantía)

Veamos la consola

Cómo usar Python

1. Como programa
2. Como consola interactiva
3. **Jupyter/Collab/IDE (ayudantía)**

Veamos la consola

Primeros códigos

- Definición de variables (case sensitive)

$$> \quad \underbrace{\texttt{a = 2}}_{\text{Asignación } \textit{siempre} \text{ de derecha a izquierda}}$$

- Toda variable tiene un tipo
  - `> a = 2 :` 'a' es un 'int' (entero)
  - `> a = 3.14:` 'a' es un 'float' (punto flotante)
  - `> a = "hola":` 'a' es un 'string' (texto)
  - `> a = True:` 'a' es un 'bool' (booleano)[2]
- Se pueden dejar comentarios con #

Veamos en consola usando la función `type`

[2] esto tendrá sentido más adelante

Aritmética

Cuál es el valor resultante de las siguientes operaciones?

- `> a = 2+2`
- `> a = 2+2*2`
- `> a = 2/2*2`
- `> a = 2**2` (elevado)
- `> a = 'ho' + 'la'`

Ojo: división entre enteros da un 'float'

Nota sobre la división

La división se entiende como aplicada exclusivamente al número precedente. Eso implica lo siguiente:

- 2/2 es $\frac{2}{2}$
- 2/2*2 es $\left(\frac{2}{2}\right) \times 2$
- 2/2/2 es $\frac{\left(\frac{2}{2}\right)}{2} = \frac{2}{2 \times 2}$

Es más fácil imaginar a/b como $a \times b^{-1}$

Nota sobre la división

La división se entiende como aplicada exclusivamente al número precedente. Eso implica lo siguiente:

- 2/2 es $\frac{2}{2}$
- 2/2*2 es $\left(\frac{2}{2}\right) \times 2$
- 2/2/2 es $\frac{\left(\frac{2}{2}\right)}{2} = \frac{2}{2 \times 2}$

Es más fácil imaginar a/b como $a \times b^{-1}$

Álgebra booleana

Keywords importantes:

- `and`, `or`, `not`
- `==`, `<=`, `>=`, `<`, `>`

En términos de precedencia, `and` es un producto y `or` es una suma:

$$\texttt{a and b or c} = (\texttt{a and b})\ \texttt{or}\ c$$

Notar que el resultado es siempre otro booleano.

Recap

- Qué es programar, pensamiento algorítmico
- Sistemas operativos
- Podemos interactuar con el PC a través de la consola (Unix)
- Concepto de compilación
- Python: lenguaje interpretado, orientado a objetos
- Usamos la consola interactiva de Python
- Tipos de variables: int, float, bool, string
- Operaciones entre variables, aritmética y álgebra

Clase 0: Programación, Lenguajes y Python

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile